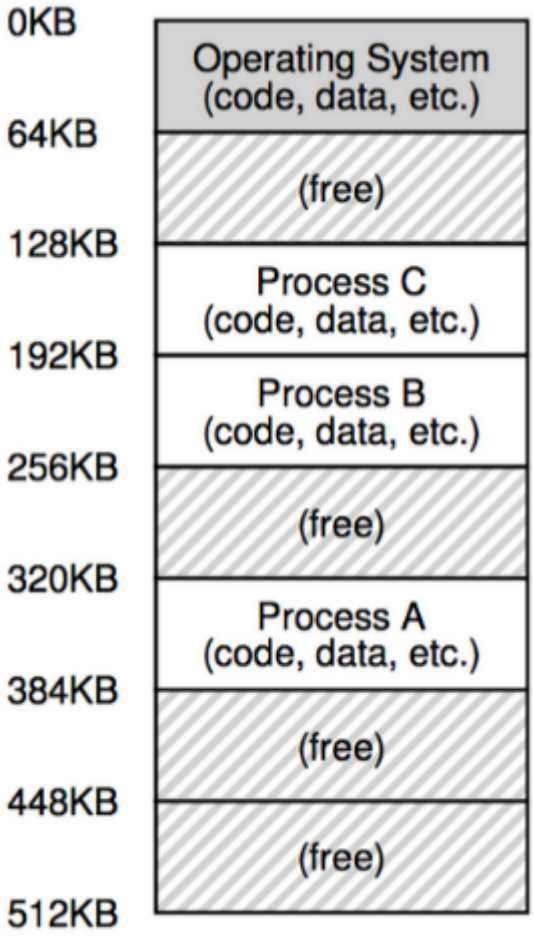
Administración de memoria

La memoria del computador está organizada como un arreglo muy grande de *bytes*. Cada byte tiene una dirección única. La introducción de la multiprogramación trajo consigo dos problemas:

- Las variables no siempre están en la misma dirección. Requisito: relocalización de variables.
- Un proceso podría leer y modificar memoria de otro. Requisito: protección de memoria.

Es evidente entonces, que con multiprogramación, no nos sirven direcciones absolutas. Así, una solución es que el compilador no genera direcciones absolutas, si no que direcciones reubicables (e.g. JMP 32+X). Esto permite utilizar un espacio de direcciones.

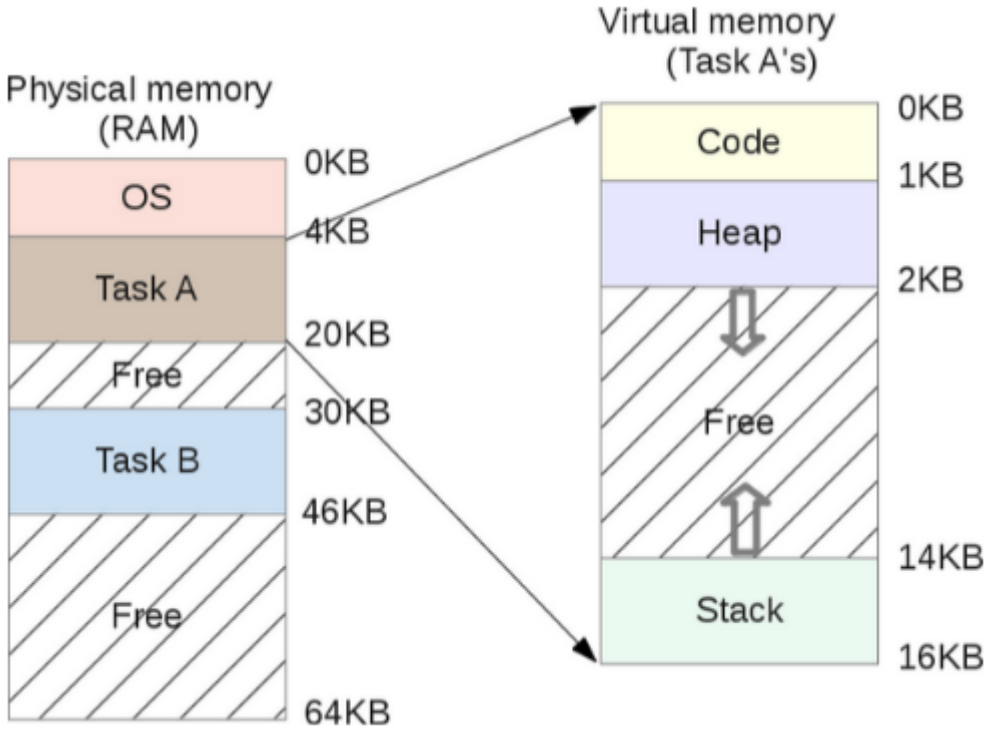


PROF

Espacio de direcciones

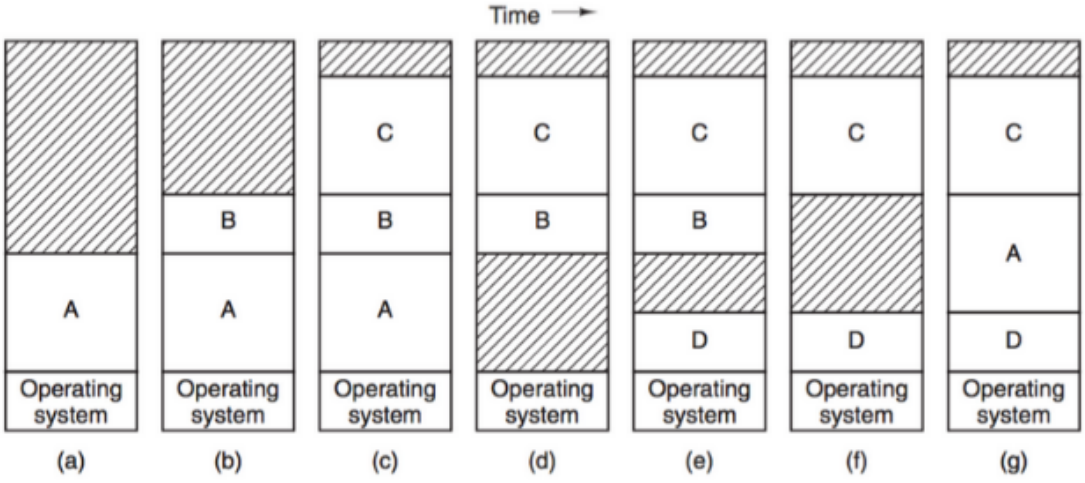
Cuando uno obtiene las direcciones de memoria de un programa, lo que recibe son direcciones virtuales! Esta abstracción se llama espacio de direcciones.

El proceso mantiene un espacio **único** y **secuencial** (lineal) de direcciones. Un proceso de 16KiB utiliza direcciones de 0 a 16383. El sistema operativo mapea estas direcciones a direcciones físicas. Esto lo hace por *hardware* en la *MMU*.



Sobrecarga de memoria: ¿qué pasa si la memoria se llena?

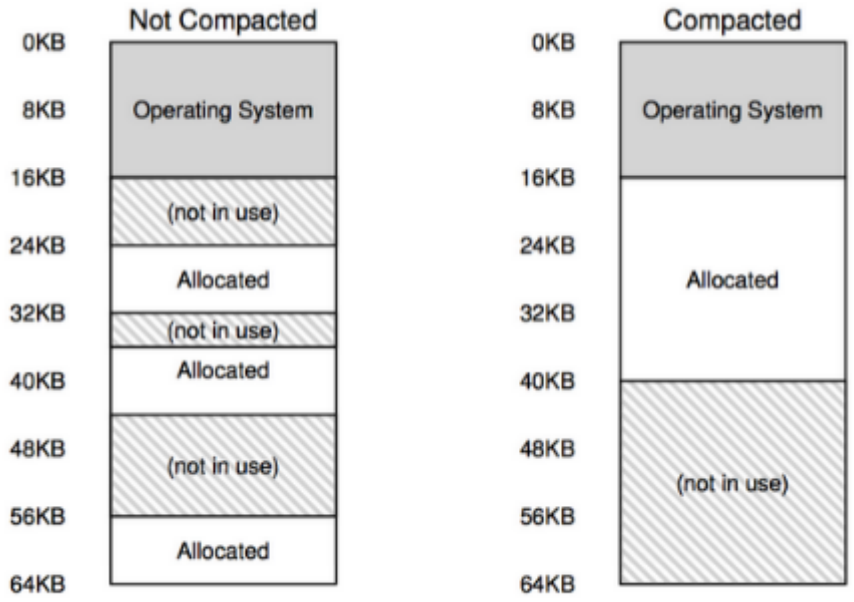
El *mid term scheduler* determina cuando hay que hacer operaciones de *swapping*. Esta operación utiliza una porción del disco, denominado espacio de *swap*. Acá se guardan imágenes de memoria de procesos. Así, los procesos pueden ser cargados y descargados de la memoria, lo que puede dejar algunos huecos...



PROF

Compactación

Una opción es realizar una operación de **compactación**, fusionando los huecos. Esto es muy costoso, ya que requiere mover todos los procesos en memoria. Algo mejor es asignar de manera inteligente los espacios de memoria para que nunca deba compactar. Esto da paso a las estrategias de alocación de memoria.



Estrategias

- *First-fit*: asigna el primer hueco que encuentre.
- *Best-fit*: asigna el hueco más pequeño que encuentre.
- *Worst-fit*: asigna el hueco más grande que encuentre.

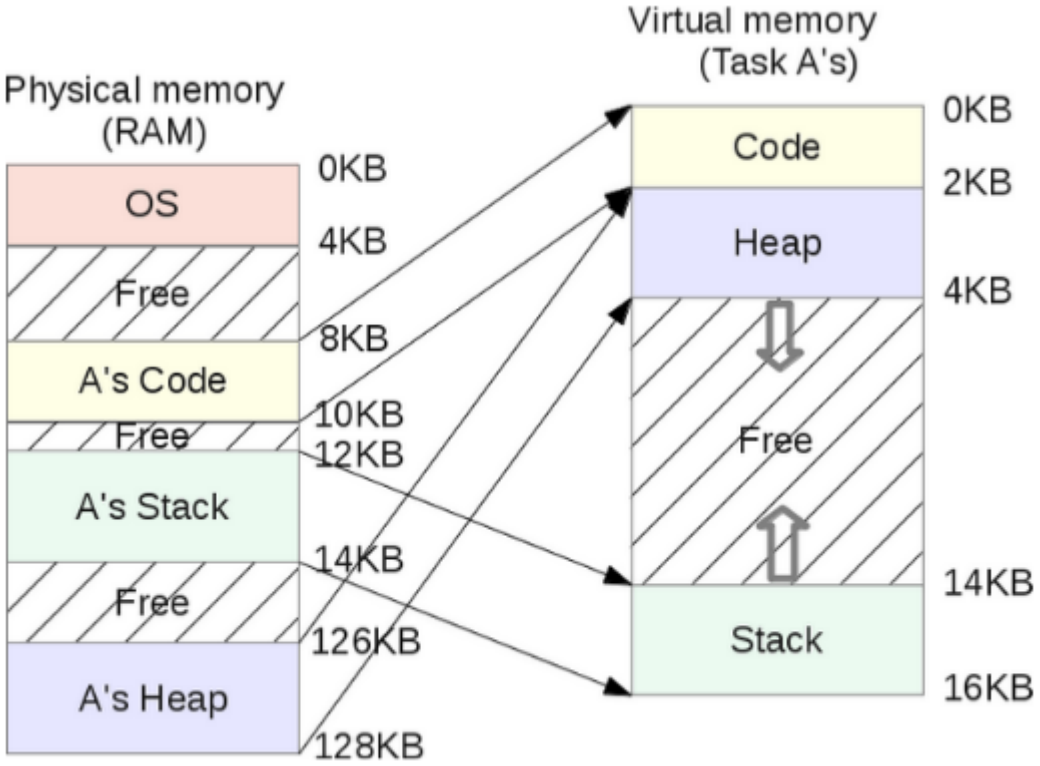
Cuando los espacios libres quedan separados se dice que hay **fragmentación**.

Segmentación

Muchas veces los procesos requieren harta memoria, en general, es difícil tener un espacio de memoria físico en donde quepa por completo. La segmentación permite dividir un proceso en segmentos más pequeños, cada uno con su propio espacio de direcciones. Esto permite que cada segmento pueda ser cargado en memoria independientemente. Los segmentos son espacios de direcciones contiguos (segmentos: code, heap, stack, ...
).

PROF

La *MMU* mapea cada segmento a un espacio de memoria físico. Queda especificado en la tabla de segmentos. Esta tabla además especifica el tamaño de memoria que se la ha asignado, por lo que, todos reciben una asignación de memoria contigua, eliminando la fragmentación externa. Sin embargo, la fragmentación interna sigue existiendo. Esta consiste en los espacios sin utilizar dentro de un segmento.



Necesito conocer el segmento y el *offset*. Con lógica de *bits* es muy fácil.

Ejemplo: ¿cuántos bit necesitamos para identificar 3 segmentos? → 2

01
10
11

13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	0	1	1	0	1	0	0	0

Segment — Offset

dirección virtual de 13 bit
* 2 bit para el segmento
* 11 bit para el offset (posición dentro del segmento)

- Dirección virtual:
  $4200 = 0b01000001101000 = 0x1068$
  - Bits de segmento: $0b01 = 1 = 0x1$
  - Bits de offset:
    $0b000001101000 = 104 = 0x068$

PROF

```
SEG_MASK = 0x3000;
OFFSET_MASK = 0xFFF;
SEG_SHIFT = 12;
virtualAddress = 4200;
segment = (virtualAddress & SEG_MASK) >> SEG_SHIFT;
offset = virtualAddress & OFFSET_MASK;
if(offset >= size[segment])
   raise(SEG_FAULT);
else
   physicalAddress = base[segment] + offset;
```

Ahora, hay que tener cuidado, porque crece en sentido contrario. Así, se agrega un bit que indica el sentido en el que crece el segmento.

La segmentación elimina la fragmentación externa (espacios libres (sin asignar, no sin ocupar) no contiguos), sin embargo, sigue existiendo fragmentación (fragmentación interna, cuando se asigna más de lo pedido). Además, es difícil anticipar el tamaño de los segmentos.

Visualización de segmentos

el comando pmap permite visualizar los segmentos de un proceso.

Paginación

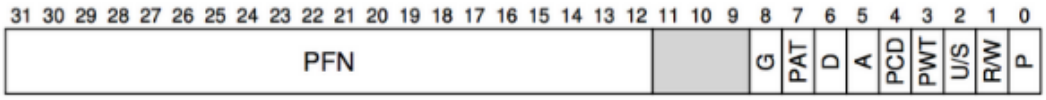
Podemos añadir una segunda idea para reducir la fragmentación: que los segmentos sean del mismo tamaño, así podemos utilizar páginas de memoria y una tabla de páginas.

- Espacio virtual: páginas.
- Espacio físico: marcos de página (*frames* o *page frames*).
- Páginas y *frames* del mismo tamaño.

Tabla de páginas

- *offset*: offset o desplazamiento dentro de la página.
- VPN: número de página virtual.
- PFN: número de marco de página físico.
- VPN es traducido a PFN por la tabla de páginas.

En una arquitectura con espacio de direcciones virtuales de 32 bits, si cada página tiene 4KiB, entonces la tabla de páginas tiene 2^20 entradas (PTE, *page table entry*). Esto genera un problema enorme: la tabla de páginas tiene un espacio de 4MiB por proceso, espacio del sistema operativo.



PTE en x86

- V, **valid bit** (no está en la imagen), indica si la página ha sido asignada
- P, **present bit**, indica si la página está en memoria o en disco.
- RW, **read/write bit**, indica si se puede escribir
- U/S, user-supervisor bit, indica si se puede acceder en *user-mode*
- PWT, PCD, PAT, G, controlan el *caching*
- A, **accessed bit**, o **reference bit** indica si la página ha sido leída
- D, **dirty bit**, indica si la página ha sido modificada

PROF

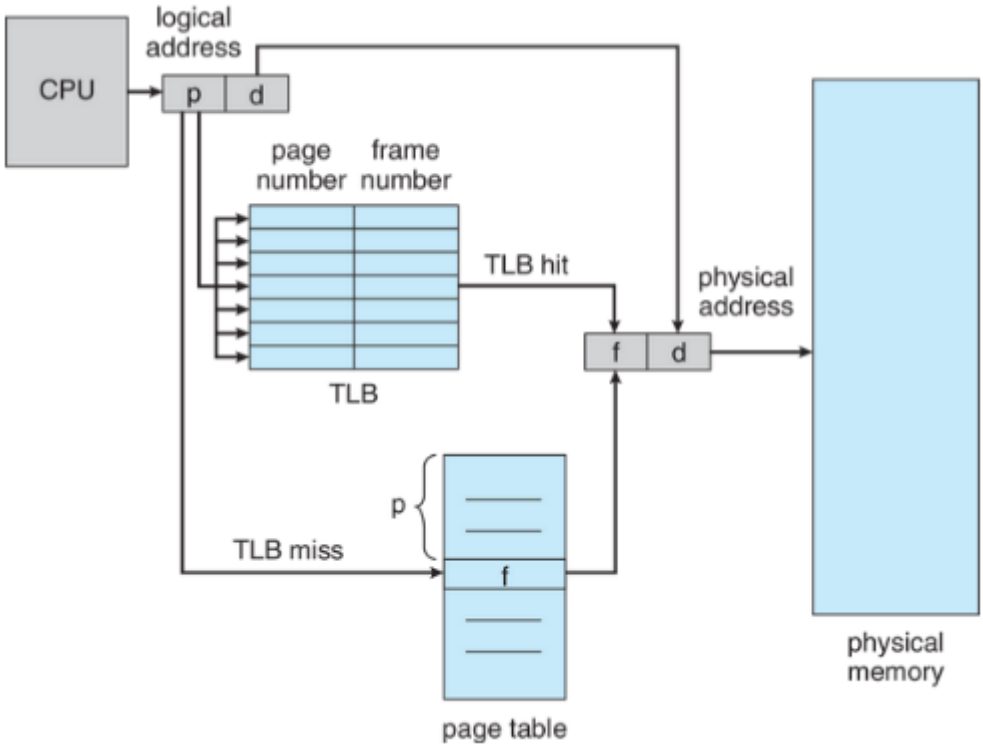
La dirección de la tabla de páginas se guarda en el PCB (*process control block*), como PTBR (*page table base register*). Así, cada acceso a memoria se convierte en dos accesos: primero se accede a la tabla de páginas, y luego a la memoria.

Paginación con TLB

Recordando que cada vez que accedo a memoria, tenemos que primero acceder a la tabla de páginas, y luego a la memoria física. Esto es muy lento, por lo que se añade un caché de páginas,

llamado TLB (*translation lookaside buffer*). Esta caché es una memoria *fully-asociative*, que guarda las últimas páginas accedidas. Así, se reduce el tiempo de acceso a memoria.

- Si la dirección está en la caché se produce un *TLB hit* y se responde inmediatamente.
- Si la dirección no está en la caché se produce un *TLB miss* y se lee desde la memoria y se actualiza el TLB.



03-05

...

Paginación multinivel